

Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Part 1:

You and your partner must share each other's Assignment 1 submission.

Part 2:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

The task is based on Faisal's name and he is attempting assignment # 1. The problem is to find the duplicates values in a tree. Additionally, if there is more than one duplicate, return the duplicate which is closest to the root node. The examples given implements Breath first search, where we traverse through the tree one level at a time

- Create 1 new example that demonstrates you understand the problem.
Trace/walkthrough 1 example that your partner made and explain it.

write example here

```
Cell In[15], line 1
  write example here
      ^
```

SyntaxError: invalid syntax

- Copy the solution your partner wrote.

```
# Definition for a binary tree node.
class TreeNode(object):
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
```

```

        self.right = right

#function to find duplicate in binary tree
def is_duplicate(root: TreeNode) -> int:
    if not root:
        return None

#initialize a set to keep track of node values seen
#initialize list to implement breadth first search traversal on binary tree
    seen = set()
    queue = [root]

#logic for checking for duplicates and tracking distance of duplicate from root
    while queue:
        current_node = queue.pop(0)
        if current_node.val in seen:
            return current_node.val
        seen.add(current_node.val)
        if current_node.left:
            queue.append(current_node.left)
        if current_node.right:
            queue.append(current_node.right)

    return -1

#tree definition example 1
root_ex1 = TreeNode(1)
root_ex1.left = TreeNode(2)
root_ex1.right = TreeNode(2)
root_ex1.left.left = TreeNode(3)
root_ex1.left.right = TreeNode(5)
root_ex1.right.left = TreeNode(6)
root_ex1.right.right = TreeNode(7)

#Test function with example 1; def'n below
is_duplicate(root_ex1)

2

[root_ex1].pop(0).val

1

```

- Explain why their solution works in your own words.
- 1. He first initialises a set() called 'seen'. When going through each node in the tree one by one. They will be stored here. Note that a set is a data type which only stores unique values
- 2. The next step is storing all the nodes of the tree in a list called 'queue'
- 3. Then implements BFS, by going through the left and right of each element and storing them in the list called 'queue'
- 4. At the start of each loop, he checks if current node is in 'seen'. If found, this will immediately return current_node.val this will make sure that the node closest to the root node
- Explain the problem's time and space complexity in your own words.

Time Complexity: The function visits each node once and stores paths for all leaves. In a balanced tree, this takes $O(N \log N)$, The good part is that the loop stops as soon as it finds the first duplicate

Space Complexity: The recursion uses $O(\log N)$ space for a balanced tree.

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

I think the DFS is implemented very well, one thing can that be improved is while creating the tree, we can create the tree using a loop. That will be a cleaner way to play with different outputs.

Part 3:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

Working on this assignment, I learned about many new data structures. In my assignment I created a function, which loops over a list and creates a tree using the helper function provided. I also learned about two ways of traversing trees. Depth first search - go through each branch till the end. Breath first search - Go thought the tree by one level at a time

The right answer of which algo is better depends on the use case. In my assignment I wanted to return the different full paths in a tree, so DFS was a better solution. In Faisal's assignment, he wanted to find the first instance of a duplicate so BFS makes more sense and is faster.

I also learnt about recursion. Which honestly still feels like magic. I understand how it works, but I want to look at more examples to truly internalise the concept.

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated

- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

▢ **Please review our [Assignment Submission Guide](#)** ▢ for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: HH:MM AM/PM - DD/MM/YYYY
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
 - This Jupyter Notebook (`assignment_2.ipynb`) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment:
`https://github.com/<your_github_username>/algorithms_and_data_structures/pull/<pr_id>`
 - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-3-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.